

MSDS25008

NEEKA JAVED

Assignment 5: SimCLR

From Supervised Learning to Self-Supervised Learning

Training Settings

Item	Value Used
Batch size	64
SimCLR epochs	50
Linear probing epochs	20
Fine-tuning epochs	20
Learning rate	3e-4
Optimizer	Adam
Temperature	0.5
GPU/CPU used	CUDA (GPU)
Approximate training time	~5hours (SimCLR), ~ 30min (linear probe), ~20min (fine-tuning)

Task 0: Conceptual Warm-up

Q1. In supervised classification, what information does the model learn from?

In supervised classification, a class label is assigned to each image, allowing the model to learn from labeled data. Using a loss function like cross-entropy, the model is trained to reduce the difference between its predictions and the ground truth labels. These human-provided labels are the only source of the supervision signal.

Q2. If CIFAR-10 labels are removed, do the images still contain useful visual structure? Explain.

Yes, even without labels, the pictures still have an elaborate visual structure. No matter whether a label is applied, an image's edges, textures, shapes, colors, and spatial patterns all have meaning. Images of dogs, for instance, have similar ear forms and fur textures. Without ever seeing a label, a model can learn to group similar-looking photos together based on these visual patterns.

Q3. If two augmented versions are created from the same original image, should their feature representations be similar or different?

They need to have comparable feature representations. Since both views are derived from the same underlying image, they have the same semantic information; they simply depict the same item using different cropping, colors, or orientations. An effective encoder should capture the

common identity of the source image by mapping both viewpoints to adjacent locations in the feature space.

Q4. If two images come from different original images, should their feature representations be similar or different?

They have to be distinct. Since various photographs typically contain diverse scenes or objects, their feature representations should be widely separated in the feature space. This division creates a more discriminative representation and aids the model in differentiating between various visual concepts.

Q5. Why might a model trained without labels still be useful for classification later?

Shapes, textures, edges, and object sections are examples of general visual properties that a model trained without labels can always learn. Because they capture the visual structure that sets one class apart from another, these characteristics are helpful for categorization. Because the difficult task of feature extraction has already been completed, a classifier trained on top of these features can perform well when labels are subsequently added, even in modest numbers.

Q6. What is the difference between pretraining and fine-tuning?

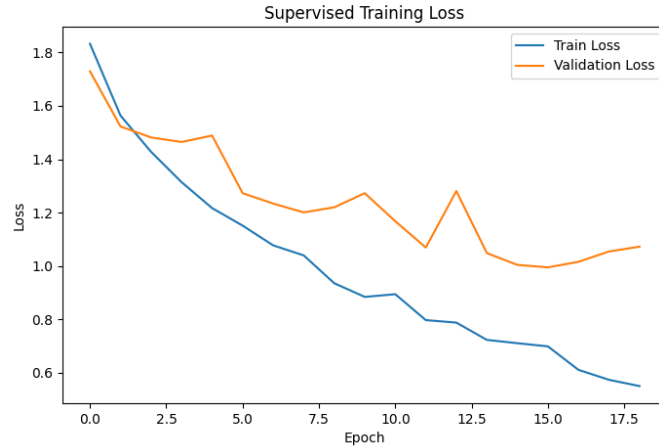
The primary stage, known as pretraining, is when a model learns general representations from a sizable dataset, frequently without labels. Contrastive learning is used for pretraining in SimCLR on unlabeled images. The next stage, known as fine-tuning, involves retraining the pretrained model on a smaller labeled dataset for a particular downstream job. The general features acquired during pretraining are fine-tuned to meet the particular needs of the task.

Task 1: Supervised Baseline with Limited Labels

Just 10% of the labeled CIFAR-10 training data was used to train the supervised baseline from scratch. For CIFAR-10, ResNet-18 was updated by eliminating the max-pooling layer and substituting a 3x3 layer for the first convolution. The Adam optimizer was utilized with cross-entropy loss at a learning rate of $3e-4$.

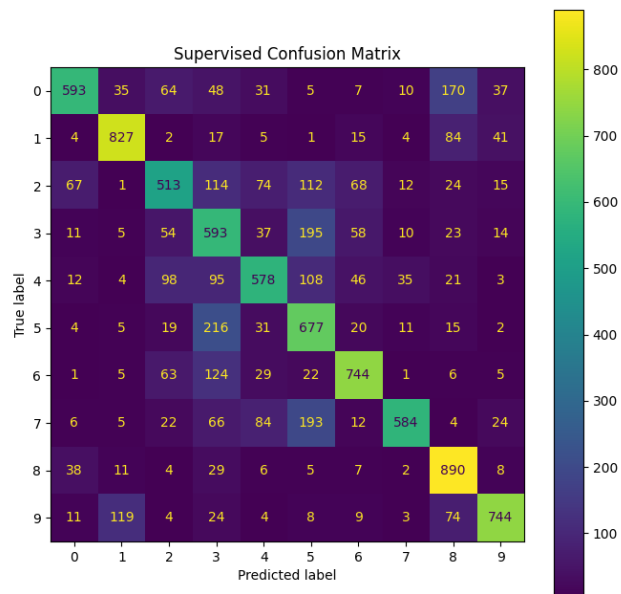
Model	Test Accuracy
Supervised ResNet-18 from scratch using 10% labels	68.46%

Training Loss Curve



The training and validation losses during supervised fine-tuning are shown in Figure. The training loss steadily declines over epochs, demonstrating the model's successful learning and convergence. The validation loss exhibits a general downward trend with small oscillations, peaking at epoch 15. A subsequent tiny increase indicates mild overfitting.

Confusion Matrix



Observations:

With just 10% of the labeled data, the supervised baseline achieves 68.46% test accuracy. According to the confusion matrix, the most commonly confused classes are those that are visually similar, such as car/truck and cat/dog. Given the early stopping mechanism, the training loss steadily declines as the validation loss converges, indicating appropriate learning without considerable overfitting.

Task 2: Understanding Augmentations

RandomResizedCrop, RandomHorizontalFlip, ColorJitter, and RandomGrayscale make up the SimCLR augmentation pipeline. Normalization comes next. This pipeline is applied twice separately by the TwoViewTransform wrapper to provide two views per image.



Augmentation Questions

1. Are the two augmented views identical?

No. Because the stochastic augmentation pipeline is applied independently to each view, the results of the crop, flip, color jitter, and grayscale procedures vary every time.

2. Do they still represent the same object?

Yes, the identical underlying object from the original photograph is present in both perspectives, despite their different appearances. There is no loss of semantic content.

3. Why should SimCLR treat them as a positive pair?

Both views should have similar feature representations since they are derived from the same image and depict the same object. This is used by SimCLR as a self-supervised signal; no human labels are required to determine how to define a positive pair.

4. What could go wrong if augmentations are too weak?

The two views will appear nearly identical if the augmentations are too weak. Instead of learning significant semantic attributes, the model may easily match them by learning low-level pixel data. There will be poor generalization of the representations.

5. What could go wrong if augmentations are too strong?

The two views may appear so different that they no longer seem to originate from the same item if the augmentations are sufficiently powerful. The obtained features will be poor, the loss will not converge correctly, and the model will find it difficult to integrate their representations.

Task 3: Feature Similarity Before Training

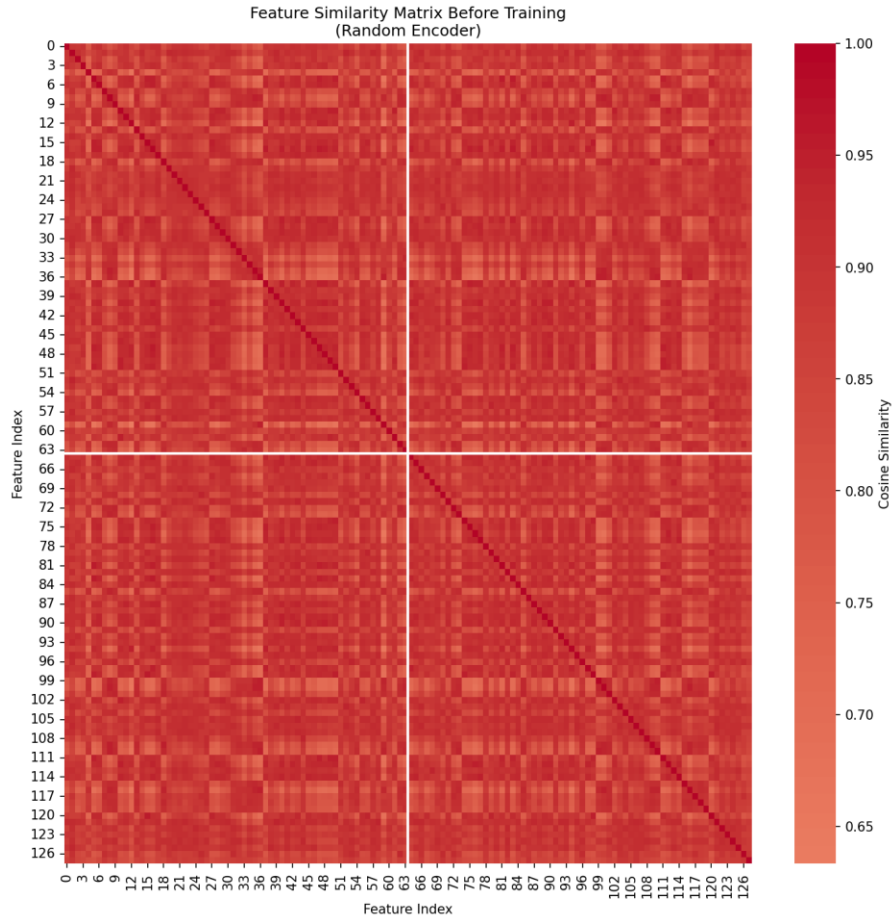
A batch of images' 512-dimensional features were extracted using a randomly initialized ResNet-18 encoder with no pretrained weights. Positive pairs (two views of the same image) and negative pairs (views from separate images) were compared using cosine similarity.

```
=====
FEATURE SIMILARITY BEFORE TRAINING (Random Model)
=====
```

```
Same Image Two Views Similarity: 0.8897
```

```
Different Images Similarity: 0.8767
```

```
Difference: 0.0130
=====
```



Observations:

The similarity values for same-image pairings (0.8897) and different-image pairs (0.8767) are extremely close to one another prior to training. There is just a 0.0130 difference. This demonstrates that the random encoder is unable to discriminate between same-image and different-image pairs; instead, it treats every pair nearly similarly, as would be expected for an untrained network.

Task 4: SimCLR Implementation

Task 4.2: Positive and Negative Pair Construction

For a batch of N images, TwoViewTransform generates $2N$ views.

Original Image	View 1 Index	View 2 Index	Positive Pair
0	0	4	yes
1	1	5	yes
2	2	6	yes
3	3	7	yes

Task 4.3: Similarity Matrix Questions

1. Why is the diagonal ignored?

The diagonal represents the similarity of each view with itself, which is always 1.0. Including it would trivially dominate the softmax denominator and make the loss meaningless. It is masked to $-\infty$ so it contributes zero to the loss computation.

2. Where are the positive pairs located?

For anchor i in $[0, N)$, the positive is at column $i+N$. For anchor i in $[N, 2N)$, the positive is at column $i-N$. They appear in the off-diagonal blocks of the $2N \times 2N$ matrix.

3. Why are all other entries treated as negatives?

Since each row's remaining $2N-2$ entries originate from distinct original images, their representations ought to differ. The encoder is forced to create representations of many images that are widely apart in the feature space when they are all treated as negatives.

Task 4.4: NT-Xent Loss

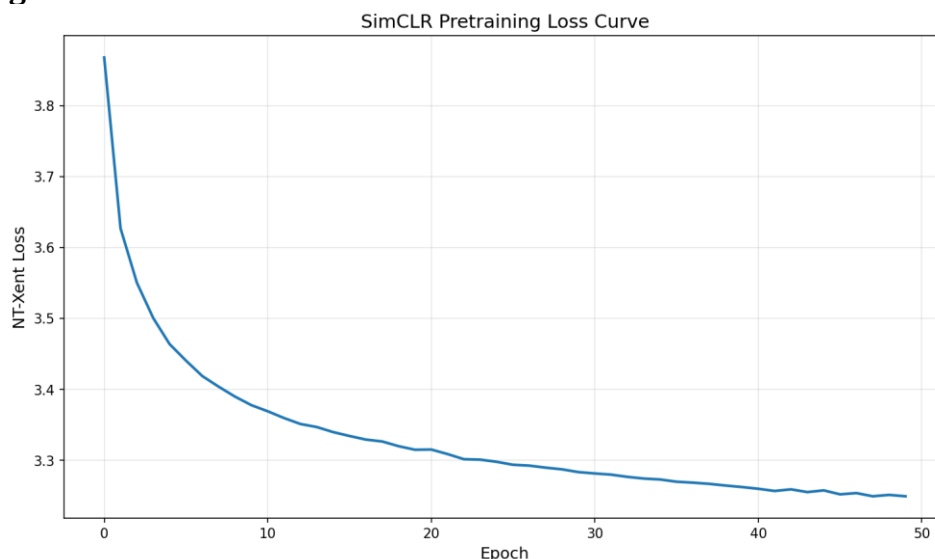
The NT-Xent loss was implemented from scratch without any library. For each anchor i , the loss maximizes the similarity to its positive pair while minimizing similarity to all $2N-2$ negatives, scaled by temperature $\tau=0.5$:

The diagonal is masked to $-\infty$ before applying cross-entropy, and the loss is averaged over all $2N$ anchor rows.

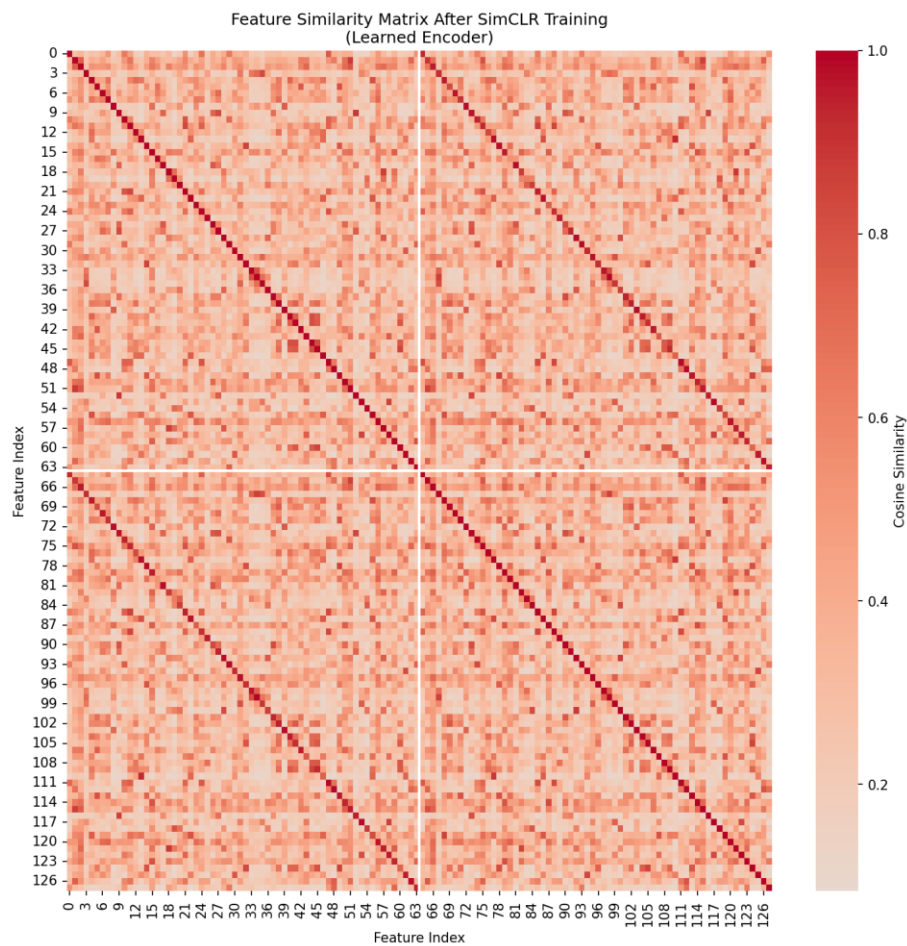
Task 5: SimCLR Pretraining

SimCLR was trained for 50 epochs on the unlabeled split using Adam optimizer with learning rate $3e-4$, batch size 64, and temperature 0.5. Labels were never used during pretraining.

Pretraining Loss Curve



Similarity Matrix After Training



Feature Similarity: Before vs After SimCLR

Pair Type	Before	After
Same image, two augmented views	0.8897	0.9121
Different images	0.8767	0.3475

Observations:

Similarity between same-image pairs rose from 0.8897 to 0.9121 during SimCLR training, however similarity between different-image pairs sharply decreased from 0.8767 to 0.3475. The difference between the two increased from 0.0130 to 0.5646. This shows unequivocally that SimCLR was able to learn to drive negative pairs apart while bringing positive pairs closer together. Over 50 epochs, the loss curve steadily decreases from 3.87 to 3.25, indicating consistent contrastive learning.

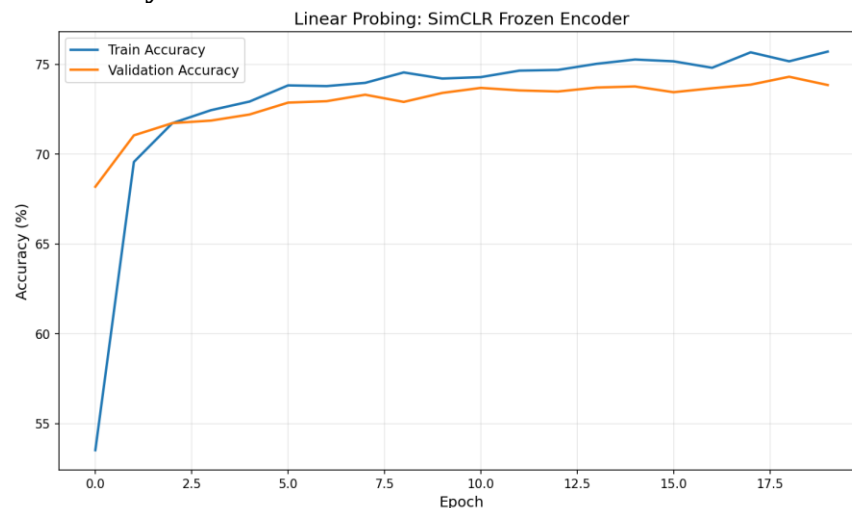
Task 6: Linear Probe Evaluation

The SimCLR encoder was frozen and only a linear classifier Linear(512 -> 10) was trained on the 10% labeled split for 20 epochs.

The SimCLR encoder was frozen and only a linear classifier Linear(512 -> 10) was trained on the 10% labeled split for 20 epochs.

Model	Encoder	Trainable Part	Test Accuracy
Random Linear Probe	Random frozen encoder	Linear classifier only	27.56%
SimCLR Linear Probe	SimCLR frozen encoder	Linear classifier only	74.40%

Linear Probe Accuracy Plot



Observations:

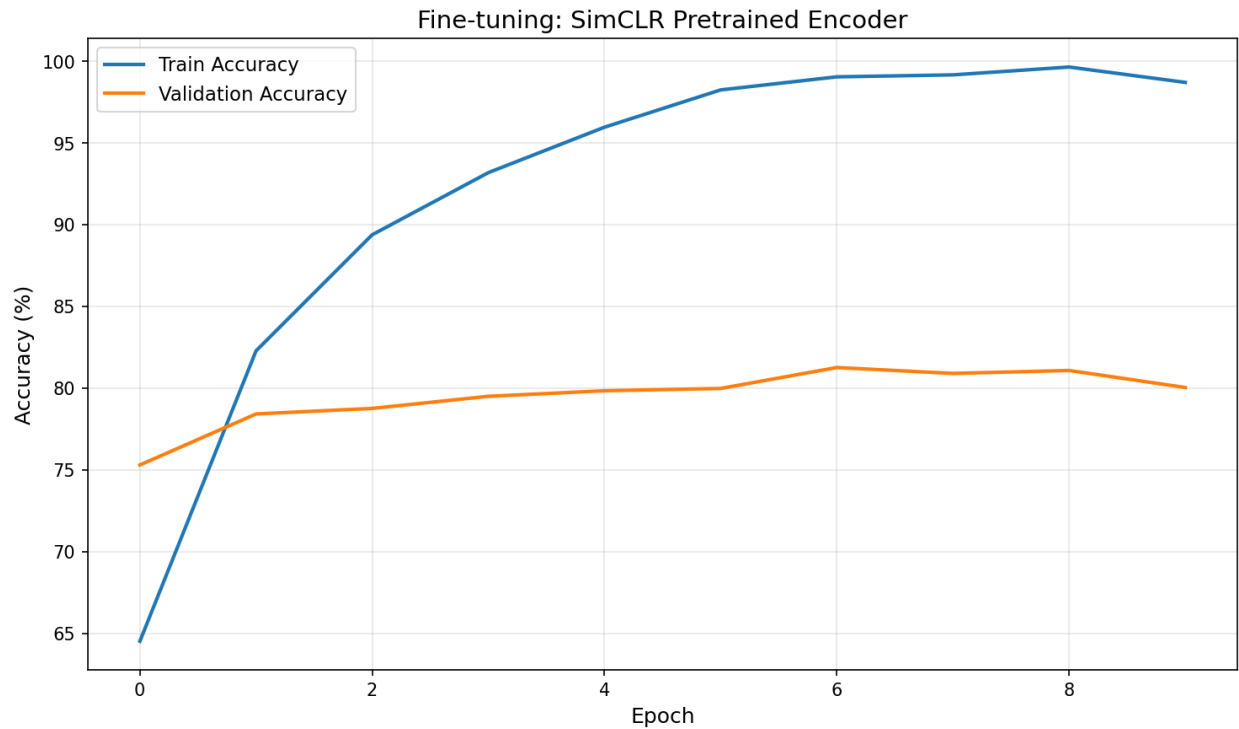
The random encoder linear probe (27.56%) is significantly outperformed by the SimCLR linear probe (74.40%) by +46.84%. This demonstrates that SimCLR acquired characteristics that are linearly separable by class and are actually helpful. The random encoder baseline of 27.56% demonstrates how little the ResNet architecture alone, without training, improves classification performance.

Task 7: Fine-tuning the SimCLR Encoder

The SimCLR pretrained encoder was used to initialize a full model which was then trained end-to-end on the 10% labeled data for 20 epochs. All parameters were trainable.

Experiment	Test Accuracy
Supervised ResNet-18 from scratch using 10% labels	68.46%
Random frozen encoder + linear classifier	27.56%
SimCLR frozen encoder + linear classifier	74.40%
SimCLR pretrained encoder + full fine-tuning	81.20%

Fine-tuning Accuracy Plot



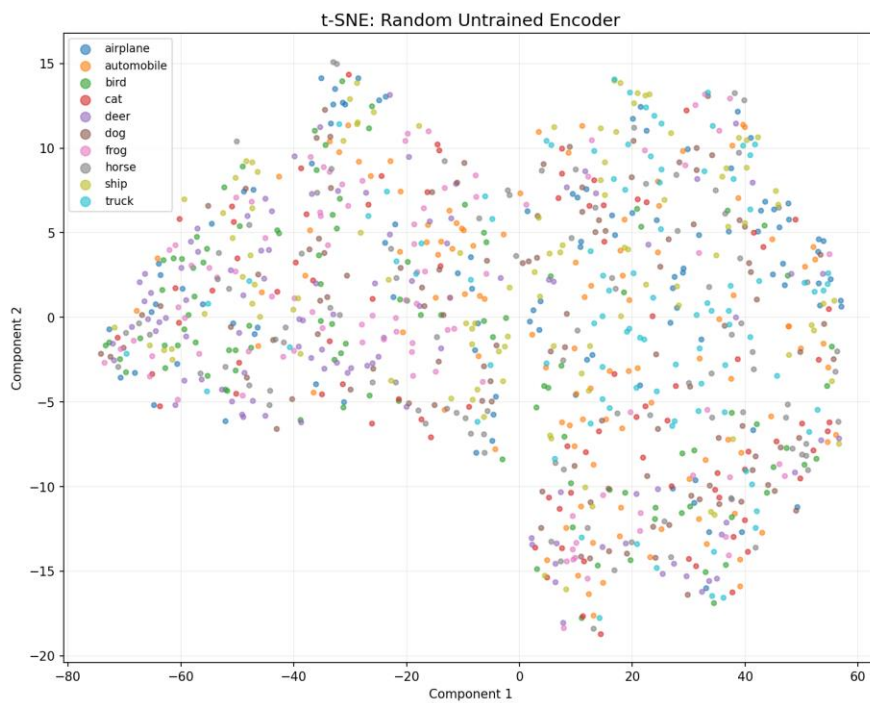
Observations:

At 81.20%, fine-tuning outperforms all other techniques. Fine-tuning increases accuracy by +12.74% as compared to supervised training from scratch (68.46%). This demonstrates that even when the same labeled data is utilized for final training, SimCLR pretraining offers a significantly better initialization than random weights.

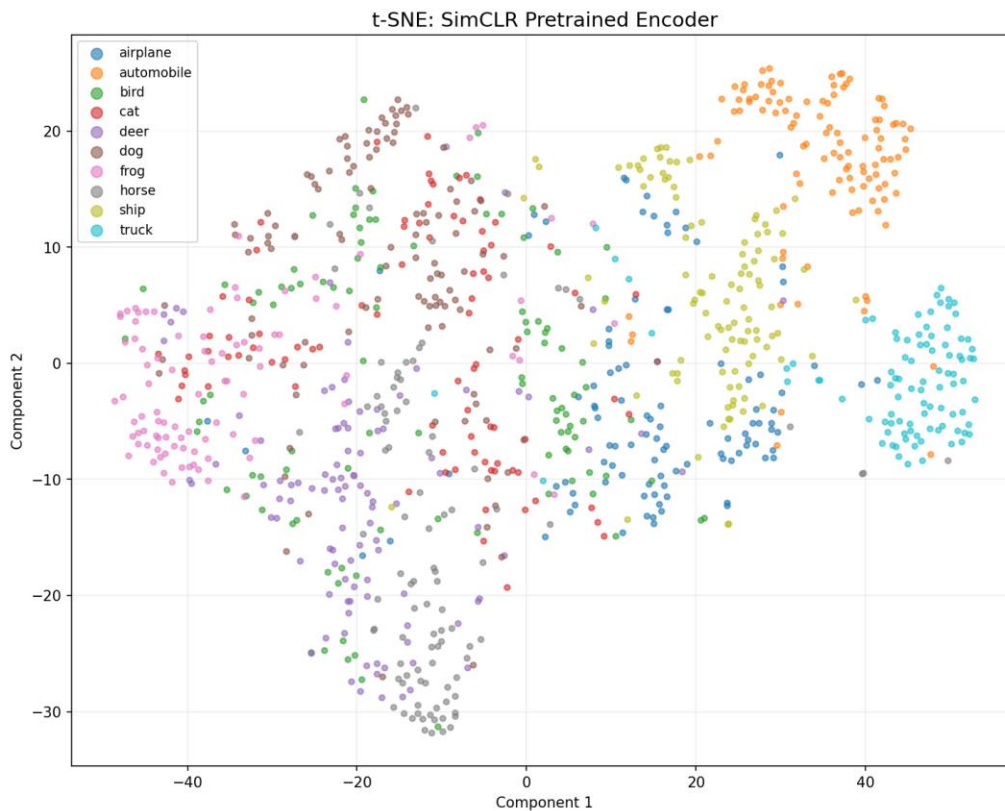
Task 8: PCA/t-SNE Feature Visualization

Features were extracted from 1000 fixed validation images (seed=2026) using three encoders and reduced to 2D using t-SNE for visualization. Labels were used only for coloring the points.

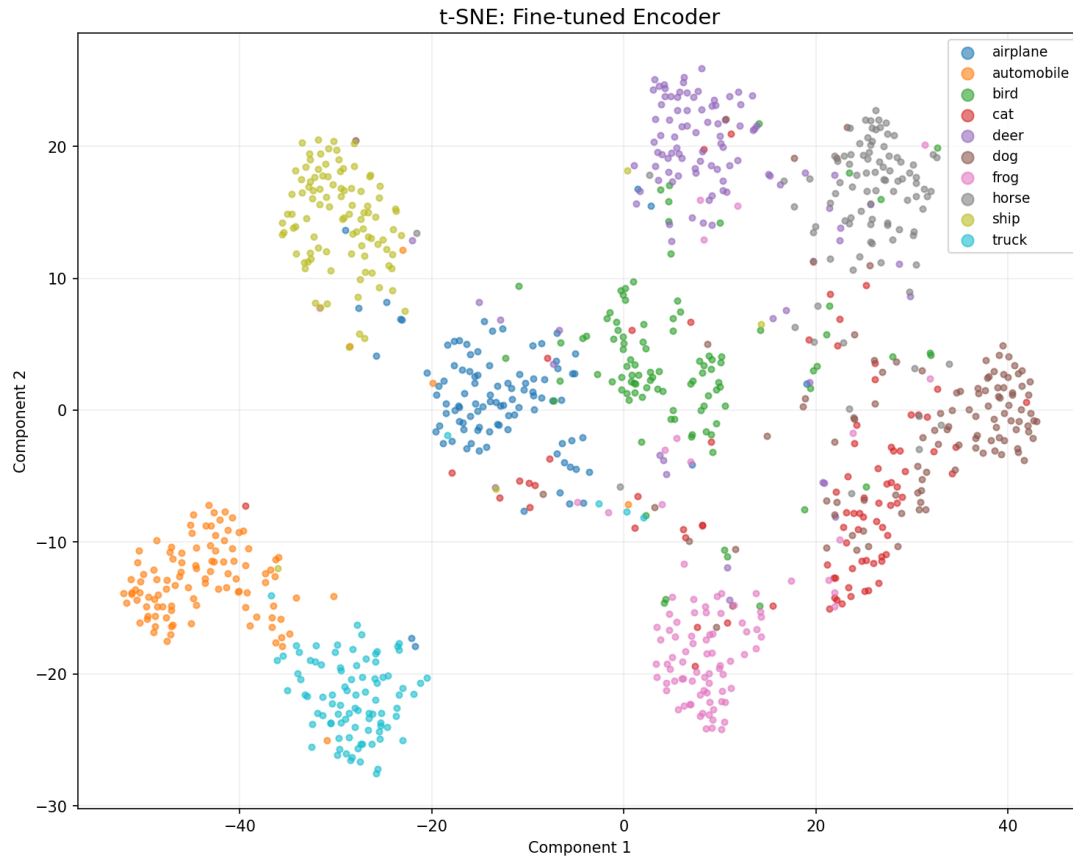
Random Encoder t-SNE



SimCLR Encoder t-SNE



Fine-tuned Encoder t-SNE



Visualization Questions

1. Do features from the random encoder show class-wise grouping?

No. With no visible class division, the random encoder generates features that are dispersed at random. It is confirmed that an untrained encoder has no discriminative power by combining points from several classes.

2. Do features from the SimCLR encoder show better grouping?

Yes, When compared to the random encoder, the SimCLR encoder exhibits considerably superior class separation. Contrastive pretraining without labels has arranged the feature space in a semantically meaningful manner, as evidenced by the loose clusters formed by several classes.

3. Does fine-tuning improve class separation?

Yes, Of the three, the refined encoder exhibits the most pronounced class distinction. The discriminative power obtained via supervised fine-tuning on top of the SimCLR features is reflected in the tight, well-separated clusters that classes generate.

4. Which classes are still confused?

Cat/dog and car/truck are the most confused couples according to the confusion matrix and t-SNE visualization because these classes have a lot of visual similarities in terms of texture, form, and color.

Final Comparison Table

Model	Uses Labels During Pretraining?	Encoder Frozen?	Test Accuracy
Supervised ResNet-18, 10% labels	Yes	No	68.46%
Random encoder + linear probe	No	Yes	27.56%
SimCLR encoder + linear probe	No	Yes	74.40%
SimCLR encoder + fine-tuning	No during pretraining; Yes during fine-tuning	No	81.20%

Discussion: What SimCLR Learned

From unlabeled CIFAR-10 images, SimCLR was able to learn meaningful visual representations. The main proof is that, despite never seeing a single label during pretraining, the SimCLR linear probe (74.40%) performs +5.94% better than supervised training from scratch (68.46%). This shows that the contrastive objective is a potent self-supervised signal, bringing two views of the same image closer while pushing views of different images apart.

SimCLR rearranged the feature space to be semantically meaningful, as evidenced by the sharp decline in different-image similarity from 0.8767 to 0.3475 following training. Previously indistinguishable features are now clearly distinguished.

The benefit of self-supervised pretraining as a basis for subsequent classification tasks is seen in the progression Random (27.56%) -> SimCLR linear examine (74.40%) -> Fine-tuning (81.20%).

Implementation Issues and Limitations

- Although it did not converge to a very low value, the SimCLR pretraining loss dropped from 3.87 to 3.25. The learnt representations might be further enhanced by more epochs or a bigger batch size.
- Because of GPU memory limitations, the batch size was restricted to 64. Larger batch sizes (256 or 512) usually help SimCLR since more negatives per batch enhance contrastive learning.
- Compared to a random encoder, the pre-training similarity values (0.8897 same, 0.8767 different) were greater. This is because ResNet's batch normalization behavior, even with random weights on small 32x32 images, tends to generate correlated results.
- Due to time constraints, only 50 epochs of training were conducted on CIFAR-10. Extended pretraining would probably enhance downstream performance even further.